CSC 370 PaperTrade, Lossless Join Proof

Dryden Bryson

November 4, 2026

Overview

Each of our eight relations is already in BCNF (see the *Functional Dependencies and Candidate Keys* document). Losslessness is a property of a *decomposition*, not of a single relation. Our relations are the result of decomposing the wide tables we would have without normalization: a **universal relation** that holds a fact together with everything it determines, in one row.

Our database stores four independent kinds of fact: trades, limit orders, daily prices and watchlist entries. Each gets its own universal relation, because putting all four in one row does not work (Section 6). This document:

1. defines the attributes, the FDs and the four universal relations U_T, U_O, U_H and U_W;
2. for the trade relation U_T, finds its candidate key, shows that it violates BCNF using anomalies from `sql/seed.sql`, and decomposes it with the BCNF decomposition algorithm into `user`, `exchange`, `stock`, `portfolio` and `transaction`, proving each step lossless;
3. does the same, more briefly, for U_O, U_H and U_W, which produce `limit_order`, `stock_price_history` and `watchlist` along with tables already found;
4. confirms each decomposition is lossless with the chase test;
5. shows why a single universal relation of every attribute cannot be decomposed losslessly into our tables;
6. gives a lossy decomposition of `limit_order` as a counterexample, with spurious tuples taken from the seed data.

1 Attributes, FDs and the Universal Relations

A is the set of every attribute in the database. `quantity`, `target_price` and `created_at` each appear in two tables with different meanings, so in A they are renamed by table: t_quantity and o_quantity (`transaction`, `limit_order`), w_target_price and o_target_price (`watchlist`, `limit_order`), p_created_at and o_created_at (`portfolio`, `limit_order`). The foreign keys `user_id`, `portfolio_id`, `ticker` and `exchange_code` mean the same thing wherever they appear, so each is one attribute.

To keep the expressions readable, we group attributes that always appear together:

$$
\begin{aligned}
N &= \{\text{name}, \text{email}, \text{cash_balance}\} && \text{(account details)}\\
Q &= \{\text{portfolio_name}, \text{p_created_at}\} && \text{(portfolio details)}\\
E &= \{\text{exchange_name}, \text{country}, \text{currency}\} && \text{(exchange details)}\\
S &= \{\text{company_name}, \text{sector}\} && \text{(stock details)}\\
H &= \{\text{opening_price}, \text{closing_price}, \text{high_price}, \text{low_price}, \text{volume}\} && \text{(daily price details)}\\
W &= \{\text{w_target_price}, \text{added_at}\} && \text{(watchlist details)}\\
T &= \{\text{transaction_type}, \text{t_quantity}, \text{price}, \text{executed_at}\} && \text{(trade details)}\\
O &= \{\text{order_type}, \text{o_target_price}, \text{o_quantity}, \text{status}, \text{o_created_at}\} && \text{(order details)}
\end{aligned}
$$

Functional Dependencies F

These are the FDs of the eight relations, from the *Functional Dependencies and Candidate Keys* document:

- FD_1: user_id $\to N$
- FD_2: email $\to$ user_id
- FD_3: portfolio_id $\to$ user_id, Q
- FD_4: user_id, portfolio_name $\to$ portfolio_id
- FD_5: exchange_code $\to E$
- FD_6: ticker $\to$ exchange_code, S
- FD_7: ticker, price_date $\to H$
- FD_8: user_id, ticker $\to W$
- FD_9: transaction_id $\to$ portfolio_id, ticker, T
- FD_{10}: order_id $\to$ portfolio_id, ticker, O

The Four Universal Relations

Each universal relation holds one kind of fact, plus every attribute that fact determines through the foreign keys:

$$
\begin{aligned}
U_T &= \{\text{transaction_id}, \text{portfolio_id}, \text{user_id}, \text{ticker}, \text{exchange_code}\} \cup N \cup Q \cup S \cup E \cup T && \text{(one row per trade)}\\
U_O &= \{\text{order_id}, \text{portfolio_id}, \text{user_id}, \text{ticker}, \text{exchange_code}\} \cup N \cup Q \cup S \cup E \cup O && \text{(one row per limit order)}\\
U_H &= \{\text{ticker}, \text{price_date}, \text{exchange_code}\} \cup S \cup E \cup H && \text{(one row per daily price)}\\
U_W &= \{\text{user_id}, \text{ticker}, \text{exchange_code}\} \cup N \cup S \cup E \cup W && \text{(one row per watchlist entry)}
\end{aligned}
$$

U_T and U_O leave out W: a user does not have to watch a stock to trade it, so a trade's row usually has no watchlist values to carry. Together the four cover all of A. The FDs that hold in each are computed in the *Minimal Basis and FD Projection* document; in each case they are exactly the FDs of F whose attributes lie in that relation.

2 Decomposing the Trade Relation U_T

Candidate Key of U_T

$$
\begin{aligned}
\{\text{transaction_id}\}^+ &\supseteq \{\text{transaction_id}, \text{portfolio_id}, \text{ticker}\} \cup T && \text{(applying } FD_9\text{)}\\
&\supseteq \ldots \cup \{\text{user_id}\} \cup Q && \text{(applying } FD_3\text{, transitivity)}\\
&\supseteq \ldots \cup N && \text{(applying } FD_1\text{, transitivity)}\\
&\supseteq \ldots \cup \{\text{exchange_code}\} \cup S && \text{(applying } FD_6\text{, transitivity)}\\
&\supseteq \ldots \cup E && \text{(applying } FD_5\text{, transitivity)}\\
&\supseteq U_T
\end{aligned}
$$

{transaction_id} is a superkey, and its only proper subset $\emptyset$ has $\emptyset^+ = \emptyset$, so it is a candidate key. No FD has transaction_id on its right-hand side. The only way to derive it is reflexivity, so every superkey must contain it. {transaction_id} is therefore the **only** candidate key of U_T.

BCNF Violations

U_T is in BCNF only if the left-hand side of every non-trivial FD is a superkey. Six of the seven FDs that hold in U_T fail this:

FD	Closure of left-hand side, within U_T	Superkey?
FD_1	$\{\text{user_id}\}^+ = \{\text{user_id}\} \cup N$	No
FD_2	$\{\text{email}\}^+ = \{\text{user_id}\} \cup N$	No
FD_3	$\{\text{portfolio_id}\}^+ = \{\text{portfolio_id}, \text{user_id}\} \cup Q \cup N$	No
FD_4	$\{\text{user_id}, \text{portfolio_name}\}^+ = \{\text{portfolio_id}, \text{user_id}\} \cup Q \cup N$	No
FD_5	$\{\text{exchange_code}\}^+ = \{\text{exchange_code}\} \cup E$	No
FD_6	$\{\text{ticker}\}^+ = \{\text{ticker}, \text{exchange_code}\} \cup S \cup E$	No
FD_9	$\{\text{transaction_id}\}^+ \supseteq U_T$	Yes

Anomalies in U_T

Each violation causes anomalies, which the seed data shows directly. The seed has 41,671 trades (41,668 inserted, plus 3 created by executed limit orders).

- **Insertion:** every row of U_T needs a transaction_id. So U_T cannot store user 165 (Priya Rossi) or user 293 (Keira Martin), who have no trades yet; the 25 portfolios that have no trades; or the TSX exchange and its stocks CNR and ATD, which have never been traded.
- **Update:** NASDAQ appears in 20,384 trades, so its name, country and currency are stored 20,384 times (FD_5). Worse, cash_balance changes with every trade (FD_1): user 173 (Jonas Haddad) has 287 trades, so his next trade would mean rewriting his balance in 287 rows, and missing one leaves U_T inconsistent.
- **Deletion:** user 330 (Yusuf Martin) has 1 trade. Deleting that row also deletes his name, email and cash balance (FD_1), and the portfolio it was made in (FD_3).

Lossless Join Test

A decomposition of R into R_1 and R_2 is lossless if and only if

$$(R_1 \cap R_2) \to R_1 \quad \text{or} \quad (R_1 \cap R_2) \to R_2 \quad \text{is in } F^+$$

That is, the shared attributes form a superkey of at least one side.

Each step of the algorithm picks a violation $X \to Y$ and splits R into $R_1 = X^+ \cap R$ and $R_2 = (R \setminus X^+) \cup X$. Then $R_1 \cap R_2 = X$, and $X \to X^+ \cap R = R_1$ by the definition of closure, so each step passes the test.

Order of the steps. We remove violations from the edges of the FD graph inward: first FD_1 and FD_5, whose right-hand sides determine nothing further, then FD_6, then FD_3. With this order each $X^+ \cap R$ is exactly one of our tables. Other orders are also lossless, but give relations that need extra splits (for example, starting with FD_3 puts N and Q in the same relation).

Step 1: Remove FD_1

- **Violation:** FD_1: $\text{user_id} \to N$, with $\{\text{user_id}\}^+ = \{\text{user_id}\} \cup N$.
- **Split:**

$$R_1 = \{\text{user_id}\} \cup N = \{\text{user_id}, \text{name}, \text{email}, \text{cash_balance}\} \implies \texttt{user}$$
$$T_1 = (U_T \setminus N) \cup \{\text{user_id}\} = \{\text{transaction_id}, \text{portfolio_id}, \text{user_id}, \text{ticker}, \text{exchange_code}\} \cup Q \cup S \cup E \cup T$$

- **Lossless:** $R_1 \cap T_1 = \{\text{user_id}\}$, and $\{\text{user_id}\}^+ \supseteq R_1$. So {user_id} is a superkey of R_1 and the split is lossless.
- FD_2 (email → user_id) now lies entirely inside `user`, where email is a candidate key. So it is no longer a violation.

Step 2: Remove FD_5

The projection of F onto T_1 has the FDs FD_3 to FD_6 and FD_9 (see the *Minimal Basis and FD Projection* document). {transaction_id} is still the only key of T_1, so FD_5 still violates BCNF.

- **Violation:** FD_5: $\text{exchange_code} \to E$, with $\{\text{exchange_code}\}^+ = \{\text{exchange_code}\} \cup E$.
- **Split:**

$$R_2 = \{\text{exchange_code}\} \cup E = \{\text{exchange_code}, \text{exchange_name}, \text{country}, \text{currency}\} \implies \texttt{exchange}$$
$$T_2 = (T_1 \setminus E) = \{\text{transaction_id}, \text{portfolio_id}, \text{user_id}, \text{ticker}, \text{exchange_code}\} \cup Q \cup S \cup T$$

- **Lossless:** $R_2 \cap T_2 = \{\text{exchange_code}\}$, and $\{\text{exchange_code}\}^+ \supseteq R_2$. So the split is lossless.

Step 3: Remove FD_6

The projection onto T_2 has FD_3, FD_4, FD_6 and FD_9. Within T_2, $\{\text{ticker}\}^+ \cap T_2 = \{\text{ticker}, \text{exchange_code}\} \cup S \neq T_2$, so FD_6 still violates BCNF.

- **Violation:** FD_6: $\text{ticker} \to \text{exchange_code}, S$.
- **Split:**

$$R_3 = \{\text{ticker}, \text{exchange_code}\} \cup S = \{\text{ticker}, \text{company_name}, \text{sector}, \text{exchange_code}\} \implies \texttt{stock}$$
$$T_3 = (T_2 \setminus (S \cup \{\text{exchange_code}\})) = \{\text{transaction_id}, \text{portfolio_id}, \text{user_id}, \text{ticker}\} \cup Q \cup T$$

- **Lossless:** $R_3 \cap T_3 = \{\text{ticker}\}$, and $\{\text{ticker}\}^+ \supseteq R_3$. So the split is lossless.

Step 4: Remove FD_3

The projection onto T_3 has FD_3, FD_4 and FD_9. Within T_3, $\{\text{portfolio_id}\}^+ \cap T_3 = \{\text{portfolio_id}, \text{user_id}\} \cup Q \neq T_3$, because it does not contain transaction_id. So FD_3 still violates BCNF (and so does FD_4).

- **Violation:** FD_3: portfolio_id $\to$ user_id, Q.
- **Split:**

$$R_4 = \{\text{portfolio_id}, \text{user_id}\} \cup Q = \{\text{portfolio_id}, \text{user_id}, \text{portfolio_name}, \text{p_created_at}\} \implies \texttt{portfolio}$$
$$R_5 = (T_3 \setminus (\{\text{user_id}\} \cup Q)) = \{\text{transaction_id}, \text{portfolio_id}, \text{ticker}\} \cup T \implies \texttt{transaction}$$

- **Lossless:** $R_4 \cap R_5 = \{\text{portfolio_id}\}$, and $\{\text{portfolio_id}\}^+ \supseteq R_4$. So the split is lossless.
- FD_4 now lies entirely inside `portfolio`, where $\{\text{user_id}, \text{portfolio_name}\}$ is a candidate key.

Result for U_T

The algorithm stops, because each of the five resulting relations is in BCNF:

Relation	Attributes	Projected FDs
`user`	user_id, name, email, cash_balance	FD_1, FD_2
`exchange`	exchange_code, exchange_name, country, currency	FD_5
`stock`	ticker, company_name, sector, exchange_code	FD_6
`portfolio`	portfolio_id, user_id, portfolio_name, p_created_at	FD_3, FD_4
`transaction`	transaction_id, portfolio_id, ticker, transaction_type, t_quantity, price, executed_at	FD_9

Each binary step is lossless, so the whole decomposition is lossless. If $U_T = R_1 \bowtie T_1$ and $T_1 = R_2 \bowtie T_2$, then $U_T = R_1 \bowtie R_2 \bowtie T_2$, and so on for every step:

$$\pi_{\texttt{user}}(U_T) \bowtie \pi_{\texttt{exchange}}(U_T) \bowtie \pi_{\texttt{stock}}(U_T) \bowtie \pi_{\texttt{portfolio}}(U_T) \bowtie \pi_{\texttt{transaction}}(U_T) = U_T$$

3 Decomposing U_O, U_H and U_W

3.1 The Limit Order Relation U_O

U_O has the same shape as U_T, with order_id and O in place of transaction_id and T. {order_id} is its only candidate key, for the same reason, and the same four steps apply:

Step	Violation	Split off	Shared attributes	Remainder
1	FD_1	`user`	{user_id}	$O_1 = U_O \setminus N$
2	FD_5	`exchange`	{exchange_code}	$O_2 = O_1 \setminus E$
3	FD_6	`stock`	{ticker}	$O_3 = O_2 \setminus (S \cup \{\text{exchange_code}\})$
4	FD_3	`portfolio`	{portfolio_id}	$\{\text{order_id}, \text{portfolio_id}, \text{ticker}\} \cup O \implies \texttt{limit_order}$

In each step the shared attributes are the left-hand side of the violation, which determines the split-off relation, so each step is lossless. The anomalies mirror U_T: for example, the 478 portfolios and 34 users with no limit orders cannot be stored in U_O.

3.2 The Daily Price Relation U_H

$\{\text{ticker}, \text{price_date}\}^+ = \{\text{ticker}, \text{price_date}, \text{exchange_code}\} \cup S \cup E \cup H = U_H$ (by FD_7, FD_6, FD_5). price_date is on no right-hand side, and within U_H nothing determines ticker, so {ticker, price_date} is the only candidate key. FD_5 and FD_6 violate BCNF: for instance, every one of the 60 daily prices of a NASDAQ stock repeats NASDAQ's name, country and currency.

Step	Violation	Split off	Shared attributes	Remainder
1	FD_5	`exchange`	{exchange_code}	$H_1 = U_H \setminus E$
2	FD_6	`stock`	{ticker}	$\{\text{ticker}, \text{price_date}\} \cup H \implies \texttt{stock_price_history}$

3.3 The Watchlist Relation U_W

$\{\text{user_id}, \text{ticker}\}^+ = U_W$ (by FD_8, FD_1, FD_6, FD_5). Since FD_2 gives email $\to$ user_id, {email, ticker} is also a superkey. Nothing in U_W determines ticker, and neither user_id nor email alone determines it either, so the candidate keys are {user_id, ticker} and {email, ticker}. FD_1, FD_2, FD_5 and FD_6 violate BCNF.

Step	Violation	Split off	Shared attributes	Remainder
1	FD_1	`user`	{user_id}	$W_1 = U_W \setminus N$
2	FD_5	`exchange`	{exchange_code}	$W_2 = W_1 \setminus E$
3	FD_6	`stock`	{ticker}	$\{\text{user_id}, \text{ticker}\} \cup W \implies \texttt{watchlist}$

Step 1 removes email with the rest of N, so FD_2 ends up inside `user`.

4 All Eight Relations

The four decompositions produce eight distinct relations. `user`, `exchange`, `stock` and `portfolio` are produced by more than one, always with the same attributes:

Relation	Produced by	Projected FDs
`user`	U_T, U_O, U_W	FD_1, FD_2
`portfolio`	U_T, U_O	FD_3, FD_4
`exchange`	U_T, U_O, U_H, U_W	FD_5
`stock`	U_T, U_O, U_H, U_W	FD_6
`stock_price_history`	U_H	FD_7
`watchlist`	U_W	FD_8
`transaction`	U_T	FD_9
`limit_order`	U_O	FD_{10}

This is exactly the schema in `sql/ddl.sql`. Every FD of F lies inside one relation, so the decomposition also preserves dependencies (see the *Minimal Basis and FD Projection* document). Each universal relation is recovered by joining its own relations, for example

$$\pi_{\texttt{exchange}}(U_W) \bowtie \pi_{\texttt{stock}}(U_W) \bowtie \pi_{\texttt{user}}(U_W) \bowtie \pi_{\texttt{watchlist}}(U_W) = U_W$$

5 Chase Tests

The chase checks a whole decomposition at once. Build a tableau with one row per relation. Put the distinguished symbol a in each column the relation contains, and a subscripted b_{ij} (row i, column j) everywhere else. Each group N, Q, E, S, H, W, T and O can be one column, because each group lies wholly inside or wholly outside every relation, and every FD's right-hand side is a union of whole groups and single attributes. The decomposition is lossless if some row becomes all a.

5.1 U_T

Columns, in order: transaction_id, portfolio_id, user_id, N, Q, ticker, exchange_code, S, E, T.

Initial tableau:

	transaction_id	portfolio_id	user_id	N	Q	ticker	exchange_code	S	E	T
`user`	b_{11}	b_{12}	a	a	b_{15}	b_{16}	b_{17}	b_{18}	b_{19}	$b_{1,10}$
`exchange`	b_{21}	b_{22}	b_{23}	b_{24}	b_{25}	b_{26}	a	b_{28}	a	$b_{2,10}$
`stock`	b_{31}	b_{32}	b_{33}	b_{34}	b_{35}	a	a	a	b_{39}	$b_{3,10}$
`portfolio`	b_{41}	a	a	b_{44}	a	b_{46}	b_{47}	b_{48}	b_{49}	$b_{4,10}$
`transaction`	a	a	b_{53}	b_{54}	b_{55}	a	b_{57}	b_{58}	b_{59}	a

Applying the FDs:

1. FD_3 (portfolio_id $\to$ user_id, Q): rows `portfolio` and `transaction` agree on portfolio_id $= a$, and `portfolio` has a in both columns. Set $b_{53} := a$ and $b_{55} := a$.
2. FD_1 (user_id $\to N$): rows `user`, `portfolio` and `transaction` now agree on user_id $= a$, and `user` has $N = a$. Set $b_{44} := a$ and $b_{54} := a$.
3. FD_6 (ticker $\to$ exchange_code, S): rows `stock` and `transaction` agree on ticker $= a$, and `stock` has a in both columns. Set $b_{57} := a$ and $b_{58} := a$.
4. FD_5 (exchange_code $\to E$): rows `exchange`, `stock` and `transaction` now agree on exchange_code $= a$, and `exchange` has $E = a$. Set $b_{39} := a$ and $b_{59} := a$.

Final tableau:

	transaction_id	portfolio_id	user_id	N	Q	ticker	exchange_code	S	E	T
`user`	b_{11}	b_{12}	a	a	b_{15}	b_{16}	b_{17}	b_{18}	b_{19}	$b_{1,10}$
`exchange`	b_{21}	b_{22}	b_{23}	b_{24}	b_{25}	b_{26}	a	b_{28}	a	$b_{2,10}$
`stock`	b_{31}	b_{32}	b_{33}	b_{34}	b_{35}	a	a	a	a	$b_{3,10}$
`portfolio`	b_{41}	a	a	a	a	b_{46}	b_{47}	b_{48}	b_{49}	$b_{4,10}$
`transaction`	a	a	a	a	a	a	a	a	a	a

The `transaction` row is now all a, so the decomposition of U_T is **lossless**. This agrees with the step-by-step proof.

5.2 U_O

The tableau is the one above with order_id, O and a `limit_order` row in place of transaction_id, T and the `transaction` row. The same four FD applications (FD_3, FD_1, FD_6, FD_5) make the `limit_order` row all a, so the decomposition is lossless.

5.3 U_H

	ticker	price_date	exchange_code	S	E	H
`exchange`	b_{11}	b_{12}	a	b_{14}	a	b_{16}
`stock`	a	b_{22}	a	a	b_{25}	b_{26}
`stock_price_history`	a	a	b_{33}	b_{34}	b_{35}	a

FD_6: rows `stock` and `stock_price_history` agree on ticker, so set $b_{33} := a$ and $b_{34} := a$. FD_5: all three rows now agree on exchange_code, so set $b_{25} := a$ and $b_{35} := a$. The `stock_price_history` row is all a: **lossless**.

5.4 U_W

	user_id	N	ticker	exchange_code	S	E	W
`user`	a	a	b_{13}	b_{14}	b_{15}	b_{16}	b_{17}
`exchange`	b_{21}	b_{22}	b_{23}	a	b_{25}	a	b_{27}
`stock`	b_{31}	b_{32}	a	a	a	b_{36}	b_{37}
`watchlist`	a	b_{42}	a	b_{44}	b_{45}	b_{46}	a

FD_1: rows `user` and `watchlist` agree on user_id, so set $b_{42} := a$. FD_6: rows `stock` and `watchlist` agree on ticker, so set $b_{44} := a$ and $b_{45} := a$. FD_5: rows `exchange`, `stock` and `watchlist` now agree on exchange_code, so set $b_{36} := a$ and $b_{46} := a$. The `watchlist` row is all a: **lossless**.

6 Why Not One Universal Relation?

Suppose instead we took a single universal relation over all of A. Three attributes are on no right-hand side of F: transaction_id, order_id and price_date. Every superkey must contain all three, and $\{\text{transaction_id}, \text{order_id}, \text{price_date}\}^+ = A$ (transaction_id gives ticker, and then FD_7 gives H), so this is the only candidate key of A.

None of our eight tables contains that key, and the chase shows that the decomposition of A into them is **lossy**. The three key columns are on no right-hand side, so no chase step ever changes them. At the start, only the `transaction` row has a under transaction_id, only `limit_order` has a under order_id, and only `stock_price_history` has a under price_date. No row can ever have a in all three, so no row becomes all a.

The reason is that a row of A would have to pair one trade with one limit order and one day's price, all for the same portfolio and stock. These are independent facts: the FDs say nothing about which trades go with which orders or which days. Joining our tables therefore builds every such combination, including ones the original relation did not have. The textbook fix (3NF synthesis) would add a ninth table holding (transaction_id, order_id, price_date), but that table would record nothing real. The better fix is not to put independent facts in one row at all, which is why we use one universal relation per kind of fact.

7 Joining the Implemented Tables

The proofs cover instances of the universal relations, where every user, portfolio, stock and exchange appears in at least one row. The implemented tables can store more than that, which is how they remove the insertion anomalies.

Tuples that have no matching tuple in another relation are **dangling tuples**, and the natural join drops them. In the seed data:

- `transaction` $\bowtie$ `portfolio` $\bowtie$ `user` $\bowtie$ `stock` $\bowtie$ `exchange` returns 41,671 rows, one per trade. The foreign keys guarantee that every trade matches. Dangling: users 165 and 293, the 25 portfolios without trades, the stocks CNR and ATD, and the TSX exchange.
- `limit_order` $\bowtie$ `portfolio` $\bowtie$ `user` $\bowtie$ `stock` $\bowtie$ `exchange` returns 1,448 rows, one per order. Dangling: 34 users, 478 portfolios, CNR, ATD and TSX.
- `stock_price_history` $\bowtie$ `stock` $\bowtie$ `exchange` returns 840 rows, one per daily price. Nothing dangles: every stock has prices and every exchange lists a stock.
- `watchlist` $\bowtie$ `user` $\bowtie$ `stock` $\bowtie$ `exchange` returns 1,045 rows, one per watchlist entry. Dangling: the 75 users with an empty watchlist.

Dangling tuples are data that the universal relation could never have held. They do not contradict losslessness: projecting any legal instance of a universal relation and joining the projections back still returns exactly that instance.

8 Counterexample: A Lossy Decomposition

To show why the test matters, decompose `limit_order` so that the shared attributes are not a superkey of either side:

$$L_1 = \{\text{portfolio_id}, \text{o_quantity}\}, \qquad L_2 = \{\text{order_id}, \text{ticker}\} \cup O$$

Then $L_1 \cap L_2 = \{\text{o_quantity}\}$, and $\{\text{o_quantity}\}^+ = \{\text{o_quantity}\}$, so neither $\{\text{o_quantity}\} \to L_1$ nor $\{\text{o_quantity}\} \to L_2$ holds. The test fails.

The seed data has exactly two orders for 28 shares, placed in different portfolios on different stocks (other columns omitted):

`order_id`	`portfolio_id`	`ticker`	`order_type`	`target_price`	`quantity`
220	212	XOM	sell	118.08	28
965	981	KO	sell	64.86	28

π_{L_1} contains $(212, 28)$ and $(981, 28)$, and π_{L_2} contains the two orders without their portfolios. Joining them on o_quantity $=$ 28 pairs every portfolio with every order:

`order_id`	`portfolio_id`	`ticker`	`quantity`	
220	212	XOM	28	original
220	981	XOM	28	**spurious**
965	212	KO	28	**spurious**
965	981	KO	28	original

Over all 1,448 orders, $\pi_{L_1}(\texttt{limit_order}) \bowtie \pi_{L_2}(\texttt{limit_order})$ has 110,928 rows: the 1,448 originals plus 109,480 spurious tuples. The join claims, for example, that order 220 was placed in two different portfolios, which even breaks FD_{10} (order_id $\to$ portfolio_id). Since the original relation cannot be recovered, the decomposition is **lossy**:

$$\pi_{L_1}(\texttt{limit_order}) \bowtie \pi_{L_2}(\texttt{limit_order}) \supsetneq \texttt{limit_order}$$